Claude | ANTHROP\C

PROGRAMA DE CAPACITACIÓN

Desarrollando con IA

Unidad 8 · Gobernanza, seguridad y uso responsable de la IA

GUÍA DEL TALLER

Una política que se cumple sola

94 minutos | Individual | Día 4 · bloque tarde

Claude Pro (incluye Claude Code)



felipecuevas.dev

Tu misión en una imagen

MISIÓN

Que tu **pedidos-web** cumpla su política de IA por configuración, no de memoria.

EL PROBLEMA

- Hoy tu repo confía en que todos se acuerden de las reglas.
- El legado guarda claves y reglas de negocio que se pueden saltar.
- Un **.env** falso basta para probar si tus reglas bloquean de verdad.

TU RUTA DE 94 MINUTOS

9' Preparación
Repo listo y en verde

→

25' 1 Auditar sin creer
auditoria-legado.md

→

30' 2 Reglas que bloquean de verdad
settings.json + guardian.sh

→

15' 3 Tu política por escrito
politica-ia.md

→

15' Tu reto
OBLIGATORIO
dependabot.yml

LAS 4 IDEAS DE HOY

C1 **Todo input es sospechoso**
Un archivo o una respuesta pueden traer errores u órdenes.

C2 **Mínimo privilegio**
Claude solo hace lo que necesita.

C3 **Tus datos, tus reglas**
Decides qué datos nunca salen de tu equipo.

C4 **Política ejecutable**
Se cumple por configuración, no de memoria.

Todo en tu **pedidos-web**, con Claude Pro. Los comandos exactos están en el README del taller.

0 Preparación

9'

Antes de auditar, deja tu repo listo y en verde.

Qué logras Tu **pedidos-web** trae lo nuevo de la Unidad 8 y un **.env** falso para probar tus reglas.

+ Se suma

- **docs/gobernanza/auditoria-legado.md** (plantilla)
- **docs/gobernanza/politica-ia.md** (plantilla)
- Un **.env** falso en la raíz, solo local

No se toca

- **README.md** · **AGENTS.md** · **REVIEW.md** · **PedidosWeb.sln**
- **src/** · **tests/** · **legado/** · **migracion/**
- **.github/** (salvo **dependabot.yml**, en tu reto) · el resto de **docs/**

⚠ Ojo con settings.json

.claude/settings.json no se reemplaza. En la etapa 2 le sumas reglas y conservas el deny de **legado/** (U4) y el hook PostToolUse (U5).

Repo de material → tu pedidos-web → tests en verde

✓ Checklist

- [ ] Pediste a Claude Code que traiga lo nuevo de la Unidad 8 desde el repo de material.
- [ ] Creaste el **.env** falso con los valores del README. Ya está en **.gitignore**.
- [ ] Los tests están en verde antes de empezar.

Valores inventados: nunca uses una clave real.

⚠ Si algo de «No se toca» cambia, vuelve al último commit.

1 Auditar sin creer

25'

Con el repo listo, miras primero lo que vas a proteger: el legado.

Qué logras Una auditoría del legado donde cada hallazgo lo verificaste tú, no solo Claude.

La idea

Todo input es sospechoso, también la respuesta de un agente. Un hallazgo sin **archivo:línea** es solo una opinión.



✕ **Así no:** copiar la lista de Claude tal cual.

✓ **Así sí:** cada Descartado lleva un argumento que otro puede revisar.

Pídele a Claude Code

Usa el subagente analista-legado para auditar **legado/src/UFrmPedidos.pas** y **legado/src/UDMPrincipal.pas**: riesgos de seguridad y reglas de negocio que se pueden saltar, cada uno con archivo:línea. No modifiques nada: completa **docs/gobernanza/auditoria-legado.md**.

Revisa tú

- ¿Cada hallazgo cita archivo:línea y esa línea dice lo que afirma?
- ¿Miraste dónde se valida cada regla, no solo qué valida?
- ¿Coincide o choca con **docs/diagnostico/riesgos.md** de la U4?

✓ Listo cuando

auditoria-legado.md tiene al menos 4 hallazgos Validados y 1 Descartado con su argumento.

2 Reglas que bloquean de verdad

30'

Ya sabes qué hay que proteger. Ahora haces que Claude Code no pueda tocarlo sin permiso.

Qué logras Reglas y un hook en tu repo que se cumplen solos, probados intentando romperlos.

La idea

Mínimo privilegio: lo sensible se bloquea o pide aprobación. Una regla sola no basta: el hook revisa cada acción antes de que ocurra.

Acción de Claude

deny	Bloquea:	leer **.env** y archivos **.pem**
ask	Pregunta:	curl, wget, Invoke-WebRequest, WebFetch y editar **migracion/golden-master.csv**
hook	guardian.sh	Frena toda acción que nombre **.env** o un **.pem**

Se ejecuta

Pídele a Claude Code

Suma a **.claude/settings.json**, sin borrar nada: deny para leer .env y *.pem; ask para curl, wget, Invoke-WebRequest, WebFetch y editar **migracion/golden-master.csv**. Crea **.claude/hooks/guardian.sh** como hook PreToolUse y muéstrame el diff antes de guardar.

Revisa tú

- ¿Siguen el deny de legado/ y el hook de la U5?
- ¿/permissions muestra las reglas nuevas?
- Intenta violar cada regla, también legado/, y anota qué pasó.

Listo cuando

Probaste cada regla y anotaste el resultado, incluido lo que NO se bloqueó (una búsqueda con grep -r).

3 Tu política por escrito

15'

Tus reglas ya funcionan. Ahora las dejas por escrito, con lo que cubren y lo que no.

Qué logras Una política honesta que cualquiera del equipo puede leer y comprobar.

La idea

Si depende de que alguien se acuerde, todavía no es política. Escribe solo lo que probaste.

✕ **Así no:** «Todo está protegido».

✓ **Así sí:** un límite concreto, con su riesgo y su mitigación.

politica-ia.md

1	Datos que nunca entran
2	Controles y si se probaron
3	Lo no cubierto
4	3 escenarios con su control
5	Línea «Uso de IA» en los PR

Pídele a Claude Code

Con mis notas de la etapa 2, completa **docs/gobernanza/politica-ia.md**: datos que nunca entran, controles y si se probaron, lo no cubierto, 3 escenarios con su control y la línea «Uso de IA» para los PR. No inventes controles que no probé.

Revisa tú

- ¿Cada control dice si lo probaste?
- ¿Lo no cubierto incluye lo que viste pasar en la etapa 2?
- ¿Un compañero la entendería sin preguntarte?

Listo cuando

politica-ia.md tiene las 5 secciones y está en main junto a **settings.json** y **guardian.sh**.

Tu reto

15' OBLIGATORIO

Protegiste lo que Claude lee y ejecuta. Falta lo que tu repo descarga.

Tu reto

Crea **.github/dependabot.yml** para que GitHub vigile las dependencias de tu **pedidos-web**: paquetes NuGet y las acciones de tus workflows. Esta vez el pedido a Claude Code lo escribes tú.

Tema nuevo · Cadena de suministro

Las dependencias también son input: hay que vigilarlas.

Cómo pensarlo

1. ¿Qué tipos de dependencias tiene tu repo?
2. ¿En qué carpeta busca GitHub cada una?
3. ¿Cómo compruebas en GitHub que Dependabot leyó tu archivo?

Tu pedido

Escríbelo aquí antes de abrir Claude Code.

CONTEXTO

QUÉ QUIERO

CÓMO SÉ QUE QUEDÓ BIEN

Revisa tú

- ¿Tu pedido nombra los dos tipos de dependencias?
- ¿Leíste el diff antes del commit?

Listo cuando

- El archivo está en main.
- En Insights › Dependency graph › Dependabot aparecen NuGet y GitHub Actions.
- Si llega un PR de Dependabot, no lo mergeas hoy: lo revisas como cualquier PR.

⚠ ¿No alcanzas en clase? Lo terminas en casa: es parte del entregable.

Lo que entregas hoy

Política de IA que se cumple sola

- [ ] **Etapa 1** auditoria-legado.md con Validados y Descartados
- [ ] **Etapa 2** settings.json + guardian.sh en main, cada regla probada
- [ ] **Etapa 3** politica-ia.md
- [ ] **Reto** dependabot.yml (tu reto)

Antes de irte

- [ ] Todo está en main, con commits claros.
- [ ] El **.env** falso sigue solo en tu equipo.
- [ ] Los tests siguen en verde.

La seguridad no depende de acordarse: se configura, se prueba y viaja con el repo.

Hoy lo comprobaste en tu propio repo.

Una pregunta para llevarte

¿Qué regla de tu equipo depende hoy de que alguien se acuerde?

Si algo falla

⚠ El hook no corre

→ En Windows, revisa que tengas Git Bash y reinicia Claude Code desde la raíz. Si sigue, pídele a Claude Code que revise cómo quedó registrado el hook.

⚠ Perdiste el deny de la U4 o el hook de la U5

→ Reemplazaste settings.json. Vuelve al último commit y suma las reglas de nuevo, sin borrar.

⚠ La regla «no hace nada»

→ Revisa /permissions. Si no aparece, pídele a Claude Code que valide el JSON de settings.json y reinicia la sesión.

Para aprender más

Etapa 1 — **Seguridad en Claude Code**
Cómo se defiende de instrucciones escondidas.
code.claude.com/docs/es/security

Etapa 2 — **Permisos**
allow, ask y deny, y qué no cubre una regla.
code.claude.com/docs/es/permissions

Etapa 2 — **Hooks**
PreToolUse, la entrada JSON y los códigos de salida.
code.claude.com/docs/es/hooks

Etapa 3 — **Uso de datos**
Qué se guarda, dónde y por cuánto tiempo.
code.claude.com/docs/es/data-usage

Tu reto — **Configurar Dependabot**
Opciones del archivo dependabot.yml.
docs.github.com/en/code-security/dependabot/working-with-dependabot/dependabot-options-reference

Fuentes revisadas el 12-nov-2026.